Recitation 4 Solutions

Problem 1

Develop a function named `fun1` that takes two arguments, `sum` and `n`, and uses a `while` loop to simulate rolling two unfair six-sided dice. The function should continue rolling the dice until a certain `sum` (summation of the values shown on the faces of these dice) is reached for `n` times. The probabilities of each face for both dice are not uniform and are given as follows: Die1: {1 - 0.1, 2 - 0.25, 3 - 0.2, 4 - 0.1, 5 - 0.25, 6 - 0.1} and Die2: {1 - 0.15, 2 - 0.15, 3 - 0.1, 4 - 0.15, 5 - 0.35, 6 - 0.1}. The function must return the number of rolls it took to reach the specified `sum` `n` times.

Test your function with `sum = 10` and `n = 4`.

```
fun1 <- function(sum, n) {

  n_trials <- 0

  n_successes <- 0

  while (n_successes < n) {

    n_trials <- n_trials + 1

    die_1 <- sample(1:6, size = 1, prob = c(0.1, 0.25, 0.2, 0.1, 0.25, 0.1))

    die_2 <- sample(1:6, size = 1, prob = c(0.15, 0.15, 0.1, 0.15, 0.35, 0.1))

    if (die_1 + die_2 == sum) {

      n_successes <- n_successes + 1
    }
  }

  n_trials
}

fun1(sum = 10, n = 4)
```

```
## [1] 39
```

Problem 2

Create a function (name it `function2`) that takes on three arguments (`n`, `mu`, and `sd`) and does the following:

- first it generates `n` observations from a normal distribution with mean `mu` and standard deviation `sd`
- subsequently, it takes random samples from these `n` observations until the summation of the selected observations exceeds 10 (you start with a random sample of size 1 and if the condition is not met, you keep increasing the sample size by 1 until the condition is satisfied)
- afterward, the function compares three mean values: the mean of the sample obtained from the preceding step (if the condition is not satisfied within 100 iterations, it uses the sample from the 100th iteration), the mean of the initial `n` observations, and the specified value `mu`
- lastly, it returns the largest mean value with the corresponding annotation/message (either "mu", "initial observations", or "sample")

Test your function with `n = 500`, `mu = 3`, and `sd = 2`

```
function2 <- function(n, mu, sd) {

  obs <- rnorm(n, mu, sd)

  for (sample_size in 1:100) {

    sample <- sample(obs, size = sample_size)

    sum <- sum(sample)

    if (sum > 10) break

  }

  means <- c("sample" = mean(sample), "initial observations" = mean(obs), "mu" = mu)

  sorted_means <- sort(means, decreasing = TRUE)

  sorted_means[1]

  }

function2(n = 500, mu = 3, sd = 2)
```

```
##   sample
## 3.522471
```

Problem 3

Create a function named `test_scores_variation` that takes two input arguments, `mean_score` and `score_sd`, and performs the following simulation: for each integer `n` between 1 and 50, it generates a sample of size `n` from a normal distribution with a mean of `mean_score` and a standard deviation of `score_sd`. For each sample, the function calculates the average test score and finds the difference between the average score and the `mean_score` argument. The `test_scores_variation` function should return a vector containing these differences.

Test your function with `mean_score = 75` and `score_sd = 10`.

```r
test_scores_variation <- function(mean_score, score_sd) {

  sapply(1:50, function(n) {

    random_scores <- rnorm(n, mean = mean_score, sd = score_sd)

    avg_score <- mean(random_scores)

    avg_score - mean_score

  })
}

test_scores_variation(mean_score = 75, score_sd = 10)
```

```
##  [1] -12.64409851   9.70199161  -5.75691013  -4.28736600  -1.86974128
##  [6]   2.93686700   3.30605257   4.33152065  -1.23431712   2.12729701
## [11]  -0.46846167   0.41788839   2.26722691   1.58314752   3.06432672
## [16]   2.13383378  -2.31369253  -0.03942969  -5.06356508  -0.64496237
## [21]   1.24021453  -2.35888903   0.83767381   1.02459248  -1.47622519
## [26]   3.04871278   2.53970353   3.21171776   3.59730338   3.37157542
## [31]   1.40653652  -2.46270541  -0.14602967   1.05530973   0.32727020
## [36]  -0.88144472  -0.04266774  -0.30452008   0.75034830  -0.34420266
## [41]   0.55788285   0.83988132  -1.28698689   0.32728714   0.50088404
## [46]  -0.55681227  -2.02808449   1.08785088   0.03232265   1.44790219
```

Problem 4

Write a function `streak_game()` that simulates flipping a fair coin repeatedly until either 3 heads in a row or 4 tails in a row occur. The function should return a list containing the full sequence of flips, a string indicating which condition was met (`"3 heads in a row"` or `"4 tails in a row"`), and the total number of flips performed.

```r
streak_game <- function() {

  sequence <- c()

  heads_streak <- 0

  tails_streak <- 0

  repeat {

    flip <- sample(c("H", "T"), size = 1)

    sequence <- c(sequence, flip)

    if (flip == "H") {

      heads_streak <- heads_streak + 1
```

```
      tails_streak <- 0

    } else {

      tails_streak <- tails_streak + 1

      heads_streak <- 0
    }


    if (heads_streak == 3) {

      winner <- "3 heads in a row"

      break
    }

    if (tails_streak == 4) {

      winner <- "4 tails in a row"

      break
    }
  }

  return(list(

    sequence = sequence,

    winner = winner,

    num_flips = length(sequence)))

}


streak_game()
```

```
## $sequence
##  [1] "H" "T" "T" "H" "H" "T" "T" "H" "T" "H" "H" "H"
##
## $winner
## [1] "3 heads in a row"
##
## $num_flips
## [1] 12
```

Problem 5

Write a function `rainy_days_sim(p)` that simulates weather: each day has probability `p` of being rainy, otherwise sunny. The simulation continues until 3 rainy days in a row occur. The function should return the number of days simulated.

Test with `p = 0.4`.

```
rainy_days_sim <- function(p) {

  consecutive_rain <- 0

  total_days <- 0

  while (consecutive_rain < 3) {

    total_days <- total_days + 1

    today <- sample(c("rainy", "sunny"), size = 1, prob = c(p, 1 - p))

    if (today == "rainy") {

      consecutive_rain <- consecutive_rain + 1

    } else {

      consecutive_rain <- 0
    }
  }

  return(total_days)

}

rainy_days_sim(0.4)
```

```
## [1] 43
```

Problem 6

Write a function `light_bulb_sim(n, p)` that simulates `n` light bulbs, each having a probability `p` of burning out at each time step. Each bulb has the same chance of burning out at each time step, and when a bulb is burned out, it is removed from the sample. You continue the simulation until all bulbs are burned out. The function should return the number of time steps before all bulbs burn out.

Test your function with `n = 6` and `p = 0.6`.

```
light_bulb_sim <- function(n, p) {

  time <- 0
```

```
  while (n > 0) {

    time <- time + 1

    outcome <- sample(c("lit", "burnt"), size = n, prob = c(1 - p, p), replace = TRUE)

    n <- sum(outcome == "lit")

  }

return(time)

}

light_bulb_sim(n = 6, p = 0.6)
```

```
## [1] 5
```

Problem 7

Write a function `simulate_quiz(num_questions, base_prob = 0.7)` that simulates a quiz where each question's probability of being answered correctly depends on the student's previous performance:

- The first question has probability `base_prob` of being answered correctly.
- If the student answers correctly, their probability for the next question increases by 0.05, up to a maximum of 0.95.
- If they answer incorrectly, their probability for the next question decreases by 0.05, down to a minimum of 0.3.
- The student passes if their final score is at least 60%.

The function should return `"pass"` or `"fail"`, and the total number of correct answers. Test your function with `num_questions = 20` (number of questions).

```
simulate_quiz <- function(num_questions, base_prob = 0.7) {

  prob <- base_prob

  correct <- 0

  for (i in 1:num_questions) {

    answer <- sample(c(TRUE, FALSE), size = 1, prob = c(prob, 1 - prob))

    if (answer) {

      correct <- correct + 1
```

```
      prob <- min(prob + 0.05, 0.95)

    } else {

      prob <- max(prob - 0.05, 0.3)
    }
  }

  score <- correct / num_questions


  outcome <- ifelse(score >= 0.6, "pass", "fail")


  return(list(result = outcome, total_correct = correct))
}

simulate_quiz(num_questions = 20)
```

```
## $result
## [1] "pass"
##
## $total_correct
## [1] 19
```

Problem 8

Write a function `random_walk(target)` that simulates a random walk starting from 0, where each step is +1 or -1 with equal probability. The function continues until the position reaches `target` or `-target`. It should return a list with the total number of steps and whether the walker reached the positive or negative target first.

Test your function with `target = 15`.

```
random_walk <- function(target) {

  position <- 0; steps <- 0

  while (abs(position) < target) {

    step <- sample(c(-1, 1), size = 1)

    position <- position + step

    steps <- steps + 1
  }

outcome <- ifelse(position == target, "positive_target", "negative_target")

  return(list(total_steps = steps, outcome = outcome))
```

```
}

random_walk(target = 15)
```

```
## $total_steps
## [1] 213
##
## $outcome
## [1] "negative_target"
```

Problem 9

Simulate a fishing game where a player tries to catch a specific type of fish:

- Each turn, they have a 20% chance of catching a "big fish" and an 80% chance of catching a "small fish".
- If they catch 5 big fish, they win immediately.
- If they catch 10 small fish without catching a big fish, they lose immediately.
- Otherwise, they keep fishing until they either:
  - Reach 5 big fish and win.
  - Accumulate 10 small fish without a big fish and lose.

The function, named `fishing_trip`, should return the number of attempts made and the outcome ("win" or "lose"). It takes no arguments but simulates one game per function call.

```
fishing_trip <- function() {

  attempt <- 0

  big_fishies <- 0

  accum_small_fishies <- 0

  outcome <- "lose"

  while (big_fishies < 5 && accum_small_fishies < 10) {

    attempt <- attempt + 1

    fishy <- sample(c("big", "small"), size = 1, prob = c(0.2, 0.8))

    big_fishies <- big_fishies + 1 * (fishy == "big")

    accum_small_fishies <- (accum_small_fishies + 1) * (fishy == "small")

## The cat() function is optional here and is used to make the output more
## readable

    cat(paste0(
```

```
      "Attempt #", attempt, "... ",

      "Total big fish: ", big_fishies, ", ",

      "Accummulated small fish: ", accum_small_fishies, "\n"))


    if (big_fishies == 5) {

      outcome <- "win"
    }
    }
  return(list("attempts" = attempt, "outcome" = outcome))
  }

fishing_trip()
```

```
## Attempt #1... Total big fish: 0, Accummulated small fish: 1
## Attempt #2... Total big fish: 0, Accummulated small fish: 2
## Attempt #3... Total big fish: 0, Accummulated small fish: 3
## Attempt #4... Total big fish: 0, Accummulated small fish: 4
## Attempt #5... Total big fish: 0, Accummulated small fish: 5
## Attempt #6... Total big fish: 0, Accummulated small fish: 6
## Attempt #7... Total big fish: 0, Accummulated small fish: 7
## Attempt #8... Total big fish: 0, Accummulated small fish: 8
## Attempt #9... Total big fish: 1, Accummulated small fish: 0
## Attempt #10... Total big fish: 1, Accummulated small fish: 1
## Attempt #11... Total big fish: 2, Accummulated small fish: 0
## Attempt #12... Total big fish: 2, Accummulated small fish: 1
## Attempt #13... Total big fish: 2, Accummulated small fish: 2
## Attempt #14... Total big fish: 2, Accummulated small fish: 3
## Attempt #15... Total big fish: 2, Accummulated small fish: 4
## Attempt #16... Total big fish: 2, Accummulated small fish: 5
## Attempt #17... Total big fish: 3, Accummulated small fish: 0
## Attempt #18... Total big fish: 3, Accummulated small fish: 1
## Attempt #19... Total big fish: 3, Accummulated small fish: 2
## Attempt #20... Total big fish: 3, Accummulated small fish: 3
## Attempt #21... Total big fish: 4, Accummulated small fish: 0
## Attempt #22... Total big fish: 4, Accummulated small fish: 1
## Attempt #23... Total big fish: 4, Accummulated small fish: 2
## Attempt #24... Total big fish: 4, Accummulated small fish: 3
## Attempt #25... Total big fish: 4, Accummulated small fish: 4
## Attempt #26... Total big fish: 4, Accummulated small fish: 5
## Attempt #27... Total big fish: 4, Accummulated small fish: 6
## Attempt #28... Total big fish: 4, Accummulated small fish: 7
## Attempt #29... Total big fish: 4, Accummulated small fish: 8
## Attempt #30... Total big fish: 4, Accummulated small fish: 9
## Attempt #31... Total big fish: 5, Accummulated small fish: 0

## $attempts
## [1] 31
##
## $outcome
```

```
## [1] "win"
```

Problem 10

Write a function named `charge_devices` that models charging two different devices (e.g., a phone and a laptop) with a portable battery pack. The function takes two arguments, `required_charge` (the target total charge in percentage for both devices combined) and `attempted_rounds` (how many rounds of attempts you want to simulate). The function should simulate charging the devices multiple times until the combined battery percentage reaches or exceeds `required_charge` in each of the `attempted_rounds`.

The charging efficiency for each device is not constant and is modeled as follows:

- Phone: {5% - 0.2, 10% - 0.3, 15% - 0.2, 20% - 0.15, 25% - 0.1, 30% - 0.05}
- Laptop: {10% - 0.3, 15% - 0.25, 20% - 0.2, 25% - 0.15, 30% - 0.1}

The function should return a vector of the total number of charge attempts it took in each round to meet or exceed the `required_charge` for all `attempted_rounds`.

Test your function with `required_charge = 40` and `attempted_rounds = 5`.

```
charge_devices <- function(required_charge, attempted_rounds) {

  phone <- list("pcts" = seq(5, 30, by = 5), "probs" = c(0.2, 0.3, 0.2, 0.15, 0.1, 0.05))

  laptop <- list("pcts" = seq(10, 30, by = 5), "probs" = c(0.3, 0.25, 0.2, 0.15, 0.1))

  attempts_per_round <- replicate(n = attempted_rounds, {

    attempts <- 0

    total_charge <- 0

    while (total_charge < required_charge) {

      phone_charge <- sample(phone$pcts, size = 1, prob = phone$probs)

      laptop_charge <- sample(laptop$pcts, size = 1, prob = laptop$probs)

      total_charge <- total_charge + (phone_charge + laptop_charge)

      attempts <- attempts + 1
    }

    attempts

    })

  names(attempts_per_round) <- paste("round", 1:attempted_rounds)

  return(attempts_per_round)
```

```
  }

charge_devices(required_charge = 40, attempted_rounds = 5)
```

```
## round 1 round 2 round 3 round 4 round 5
##       1       3       2       2       2
```

Problem 11

Create a function `stock_price_simulator` that simulates daily stock price changes over `days`. Each day, the stock has a probability `p_up` of increasing in value, with the increase drawn from a normal distribution with mean `mean_up` and standard deviation 1. If the stock decreases, the change is drawn form a normal distribution with mean `mean_down` and standard deviation 1. The initial stock price is given by `start_price`.

The function should accept parameters: `days`, `start_price`, `mean_up`, `mean_down`, and `p_up`. It should return a dataframe with three columns:

- `Day` - The day number
- `Price` - The stock price at the end of each day
- `Change` - The daily gain or loss

Test your function with `days` = 30, `start_price` = 100, `mean_up` = 4, `mean_down` = -5, and `p_up` = 0.55.

```
stock_price_simulator <- function(days, start_price, mean_up, mean_down, p_up) {

  day_means <- sample(c(mean_up, mean_down), size = days, prob = c(p_up, 1 - p_up), replace = TRUE)

  day_changes <- sapply(day_means, function(day_mean) rnorm(1, mean = day_mean, sd = 1))

  day_prices <- cumsum(c(start_price, day_changes))[-1]

  data.frame("Day" = 1:days, "Price" = day_prices, "Change" = day_changes)

}

stock_price_simulator(days = 30, start_price = 100, mean_up = 4, mean_down = -5, p_up = 0.55)
```

```
##    Day     Price    Change
## 1    1 103.87908  3.879076
## 2    2  99.70630 -4.172778
## 3    3  93.85007 -5.856232
## 4    4  88.68780 -5.162262
## 5    5  81.99771 -6.690089
## 6    6  85.59094  3.593226
## 7    7  87.75200  2.161056
## 8    8  82.27764 -5.474355
## 9    9  79.07776 -3.199879
## 10  10  73.09697 -5.980793
## 11  11  69.25286 -3.844109
```

```
## 12  12  74.04248  4.789618
## 13  13  79.24507  5.202594
## 14  14  74.77343 -4.471645
## 15  15  78.67459  3.901159
## 16  16  72.32262 -6.351967
## 17  17  66.13791 -6.184715
## 18  18  70.47056  4.332652
## 19  19  74.22709  3.756528
## 20  20  78.25655  4.029460
## 21  21  72.35124 -5.905303
## 22  22  66.49079 -5.860448
## 23  23  61.05028 -5.440511
## 24  24  54.15146 -6.898822
## 25  25  59.27038  5.118918
## 26  26  54.94183 -4.328549
## 27  27  48.57692 -6.364905
## 28  28  52.50854  3.931613
## 29  29  47.50999 -4.998544
## 30  30  42.69657 -4.813422
```